

Ejercicio Trigger

Casos de uso: auditoría de cambios de precios o salarios, fecha de última modificación automática, normalizar correos a minúsculas, mantener un historial, bloquear borrados de registros cerrados.

Riesgos y buenas prácticas:

- Son “invisibles”: un desarrollador puede no saber que un UPDATE dispara otras escrituras. Documentarlos y nombrarlos con prefijo (trg_tabla_momento_evento).
- Mantenerlos cortos: la lógica pesada de negocio va en procedimientos, no en triggers.
- Evitar cadenas de triggers que disparan otros triggers (difícil de depurar, riesgo de ciclos).
- Se ejecutan por cada fila (FOR EACH ROW): un UPDATE de 100.000 filas ejecuta el trigger 100.000 veces.
- En MySQL un trigger no puede modificar la misma tabla que lo disparó (error 1442); para cambiar la fila actual se usa SET NEW.col en un BEFORE.

Laboratorio:

Contexto empresarial

La **Biblioteca Universitaria “Leer+”** presta libros a sus socios. Hoy todo se controla con una hoja de cálculo y hay tres problemas:

- Nadie sabe con certeza quién tiene libros vencidos, y cada funcionario calcula los días de retraso a su manera.
- Se prestan libros que no hay en estantería y socios inactivos siguen llevando libros.
- Cuando un socio devuelve un libro, a veces el funcionario olvida actualizar el inventario.

La dirección decidió migrar a una base de datos y te pide implementar la lógica **dentro de la base de datos**, porque el sistema lo usarán el mostrador, una app móvil y un tablero de indicadores.

Reglas de negocio

1. Un socio **inactivo** no puede recibir préstamos.
2. Un socio puede tener **como máximo 3 préstamos activos** (sin devolver).
3. Solo se presta un libro si tiene **ejemplares disponibles**.
4. Un préstamo dura entre **1 y 30 días**.
5. Al prestar, los ejemplares disponibles bajan en 1; al devolver, suben en 1.
6. Todo préstamo y toda devolución quedan registrados en historial_prestamos.

Modelo de datos

Tabla	Columnas
socios	id (PK), nombre, email (único), activo
libros	id (PK), titulo, autor, ejemplares_totales, ejemplares_disponibles (≥ 0)
prestamos	id (PK), socio_id (FK), libro_id (FK), fecha_prestamo, fecha_limite, fecha_devolucion (NULL = no devuelto)
historial_prestamos	id (PK), prestamo_id (FK), accion (PRESTAMO / DEVOLUCION), detalle, usuario, fecha

Script de tablas — MySQL 8

```
CREATE DATABASE IF NOT EXISTS biblioteca_db;
```

```
USE biblioteca_db;
```

```
CREATE TABLE socios (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  nombre VARCHAR(100) NOT NULL,
```

```
  email VARCHAR(100) NOT NULL UNIQUE,
```

```
  activo BOOLEAN NOT NULL DEFAULT TRUE
```

```
);
```

```
CREATE TABLE libros (
```

```
id          INT AUTO_INCREMENT PRIMARY KEY,
titulo      VARCHAR(150) NOT NULL,
autor       VARCHAR(100) NOT NULL,
ejemplares_totales  INT NOT NULL CHECK (ejemplares_totales >= 0),
ejemplares_disponibles INT NOT NULL CHECK (ejemplares_disponibles >= 0)
);
```

```
CREATE TABLE prestamos (
id          INT AUTO_INCREMENT PRIMARY KEY,
socio_id    INT NOT NULL,
libro_id     INT NOT NULL,
fecha_prestamo  DATE NOT NULL,
fecha_limite   DATE NOT NULL,
fecha_devolucion DATE NULL,
CONSTRAINT fk_prestamos_socio FOREIGN KEY (socio_id) REFERENCES socios(id),
CONSTRAINT fk_prestamos_libro FOREIGN KEY (libro_id) REFERENCES libros(id)
);
```

```
CREATE TABLE historial_prestamos (
id          INT AUTO_INCREMENT PRIMARY KEY,
prestamo_id INT NOT NULL,
accion      VARCHAR(20) NOT NULL,
detalle     VARCHAR(255),
usuario     VARCHAR(100) NOT NULL,
fecha       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_historial_prestamo FOREIGN KEY (prestamo_id) REFERENCES
prestamos(id)
```

);

Script de tablas — PostgreSQL

CREATE TABLE socios (

id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

email VARCHAR(100) NOT NULL UNIQUE,

activo BOOLEAN NOT NULL DEFAULT TRUE

);

CREATE TABLE libros (

id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

titulo VARCHAR(150) NOT NULL,

autor VARCHAR(100) NOT NULL,

ejemplares_totales INT NOT NULL CHECK (ejemplares_totales >= 0),

ejemplares_disponibles INT NOT NULL CHECK (ejemplares_disponibles >= 0)

);

CREATE TABLE prestamos (

id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

socio_id INT NOT NULL REFERENCES socios(id),

libro_id INT NOT NULL REFERENCES libros(id),

fecha_prestamo DATE NOT NULL,

fecha_limite DATE NOT NULL,

fecha_devolucion DATE NULL

);

CREATE TABLE historial_prestamos (

```

id      INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
prestamo_id INT      NOT NULL REFERENCES prestamos(id),
accion  VARCHAR(20) NOT NULL,
detalle VARCHAR(255),
usuario VARCHAR(100) NOT NULL,
fecha   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

Datos de prueba (funcionan igual en ambos motores)

Las fechas son relativas al día en que se ejecuta el script, para que siempre haya préstamos vencidos.

```
INSERT INTO socios (nombre, email, activo) VALUES
```

```
('Camila Rojas', 'camila@correo.com', TRUE),
```

```
('Andrés Díaz', 'andres@correo.com', TRUE),
```

```
('Valentina Ríos', 'valentina@correo.com', FALSE), -- inactiva
```

```
('Mateo Castro', 'mateo@correo.com', TRUE); -- ya tiene 3 préstamos activos
```

```
INSERT INTO libros (titulo, autor, ejemplares_totales, ejemplares_disponibles)
VALUES
```

```
('Cien años de soledad', 'Gabriel García Márquez', 3, 2),
```

```
('Clean Code', 'Robert C. Martin', 2, 0), -- agotado
```

```
('Fundamentos de bases de datos', 'Abraham Silberschatz', 4, 3),
```

```
('El principito', 'Antoine de Saint-Exupéry', 2, 2);
```

```
INSERT INTO prestamos (socio_id, libro_id, fecha_prestamo, fecha_limite,
fecha_devolucion) VALUES
```

```
(1, 1, CURRENT_DATE - INTERVAL '40' DAY, CURRENT_DATE - INTERVAL '26' DAY,
CURRENT_DATE - INTERVAL '27' DAY), -- devuelto
```

(2, 2, CURRENT_DATE - INTERVAL '20' DAY, CURRENT_DATE - INTERVAL '6' DAY, NULL), -- vencido hace 6 días

(4, 2, CURRENT_DATE - INTERVAL '18' DAY, CURRENT_DATE - INTERVAL '4' DAY, NULL), -- vencido hace 4 días

(4, 1, CURRENT_DATE - INTERVAL '5' DAY, CURRENT_DATE + INTERVAL '9' DAY, NULL), -- al día

(4, 3, CURRENT_DATE - INTERVAL '2' DAY, CURRENT_DATE + INTERVAL '12' DAY, NULL); -- al día

Requerimientos

R1 — Vista. Crear v_prestamos_vencidos que muestre, para cada préstamo **no devuelto** cuya fecha límite ya pasó: id del préstamo, nombre del socio, email, título del libro, fecha límite y **días de retraso**.

R2 — Procedimiento almacenado. Crear sp_prestar_libro que reciba el socio, el libro y el número de días, y devuelva el id del préstamo creado en un parámetro de salida. Debe:

1. Validar las reglas de negocio 1 a 4 y detenerse con un mensaje claro si alguna no se cumple.
2. Crear el préstamo con fecha de hoy y fecha límite = hoy + días.
3. Descontar un ejemplar disponible.
4. Registrar la acción PRESTAMO en historial_prestamos.
5. Garantizar que, si algo falla, no quede nada a medias.

R3 — Trigger. Crear un trigger sobre prestamos que, cuando un préstamo pase de “no devuelto” a “devuelto” (se asigna fecha_devolucion), automáticamente:

1. Sume un ejemplar disponible al libro.
2. Registre la acción DEVOLUCION en historial_prestamos, indicando los días de retraso (0 si llegó a tiempo).
3. No haga nada si se actualiza otra columna o si el préstamo ya estaba devuelto.

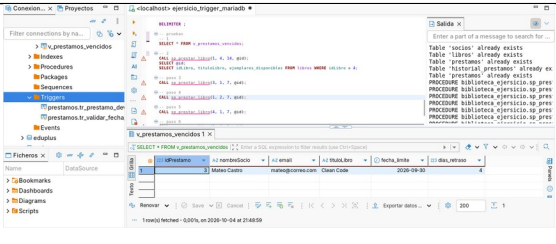
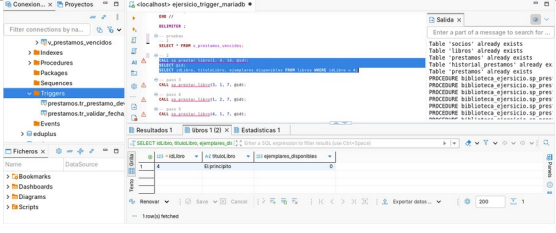
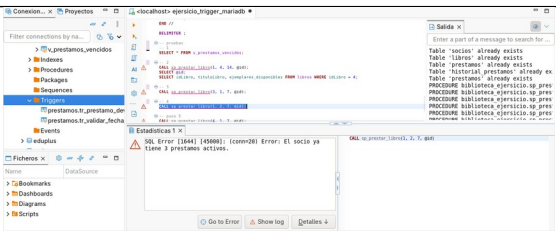
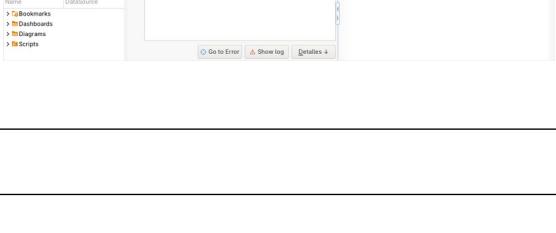
Reto opcional (para quien termine): un trigger BEFORE UPDATE que impida registrar una fecha_devolucion anterior a la fecha_prestamo.

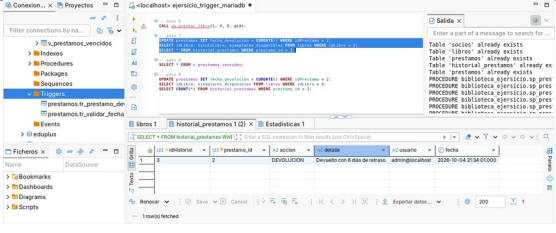
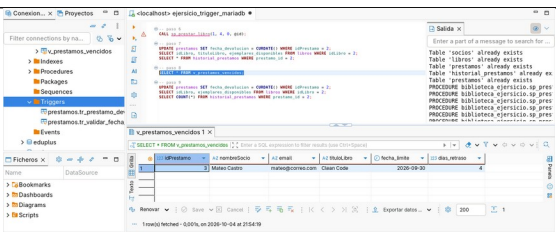
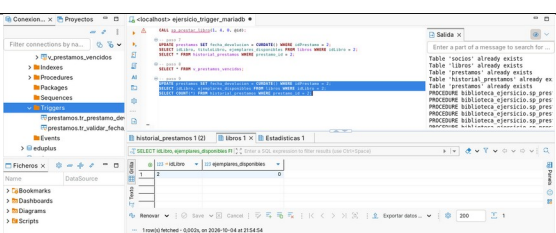
Requisitos específicos por motor

Aspecto	Si trabajas en MySQL 8	Si trabajas en PostgreSQL
---------	------------------------	---------------------------

		14+
Días de retraso	Usa DATEDIFF(fecha1, fecha2)	Resta fechas directamente: fecha1 - fecha2 devuelve un entero
Sumar días	DATE_ADD(CURRENT_DATE, INTERVAL p_dias DAY)	CURRENT_DATE + p_dias
Delimitadores	DELIMITER // antes y DELIMITER ; después de procedimientos y triggers	Cuerpo entre \$\$... \$\$, LANGUAGE plpgsql
Errores de negocio	SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = ...	RAISE EXCEPTION '...'
Atomicidad	Transacción explícita + EXIT HANDLER con ROLLBACK	Basta con lanzar la excepción: la llamada se revierte completa
Id generado	LAST_INSERT_ID()	RETURNING id INTO ...
Trigger	Un solo CREATE TRIGGER ... FOR EACH ROW BEGIN ... END	Función RETURNS TRIGGER + CREATE TRIGGER ... EXECUTE FUNCTION
Concatenar texto	CONCAT('a', x)	'a' x o CONCAT('a', x)

Pruebas que debe superar su solución (ejecutar en este orden)

#	Acción	Resultado esperado	Resultado obtenido
1	Consultar la vista	2 filas: Andrés (6 días) y Mateo (4 días)	
2	Prestar a Camila (1) “El principito” (4) por 14 días	Préstamo creado (id 6); El principito queda con 1 disponible	
3	Prestar a Valentina (3) cualquier libro	Error: socio inactivo	
4	Prestar a Camila (1) “Clean Code” (2)	Error: no hay ejemplares disponibles	
5	Prestar a Mateo (4) cualquier libro	Error: ya tiene 3 préstamos activos	
6	Prestar con 0 días	Error: días fuera de rango	

7	Registrar la devolución del préstamo 2 con fecha de hoy	Clean Code pasa a 1 disponible; historial con DEVOLUCION y 6 días de retraso	
8	Consultar la vista otra vez	Solo queda Mateo	
9	Repetir el mismo UPDATE del paso 7	No cambia el inventario ni agrega historial	

Preguntas de análisis (responder en comentarios del script)

1. ¿Por qué el reporte de vencidos es una vista y no una tabla que se llena cada noche? ¿En qué caso preferirías una vista materializada (PostgreSQL)?
2. ¿Por qué el préstamo es un procedimiento y no un trigger BEFORE INSERT sobre prestamos? Menciona al menos una ventaja y una desventaja de cada opción.
3. ¿Por qué la devolución se automatizó con un trigger y no dentro de un procedimiento sp_devolver_libro? ¿Qué riesgo aparecería si alguien hace el UPDATE directo sin trigger?
4. ¿Qué pasaría en tu procedimiento si dos funcionarios prestan al mismo tiempo el último ejemplar de un libro? ¿Qué línea lo evita?
5. Nombra dos cambios que tuviste que hacer (o tendrías que hacer) para llevar tu código al otro motor.

